

Software Development Fundamentals (SDF) – I

ODD 2023



Lecture 27-28
Week 10

**Jaypee Institute of Information Technology
(JIIT),
Noida**

Outline – Lecture 20-21

- 1. Introduction to Functions in C**
- 2. Definition and Types of Function**
- 3. Declaration and Prototypes of Functions**
- 4. Implementation of Function in various problems**
 - A) To find out square of a number using a function**
 - B) To calculate and print the squares of the integers from 1 to 10**
 - C) Finding the maximum of three integers**
 - D) Check Whether a Number is Prime or Not**

Functions in C

- A function is a self-contained block of statements that perform a coherent task of some kind.
- Every C program can be thought of as a collection of these functions.
- The idea is to put some commonly or repeatedly done task together and make a function so that instead of writing the same code again and again for different inputs, we can call the function.

```
#include
<stdio.h> main( )
{
message( ) ;
printf ( "\n Hello I am in main!" ) ;
}
message( )
{
printf ( "\n Hi I am in message" ) ;
}
```

Output:

```
Hi I am in message
Hello I am in
main!
```

- **main()** itself is a function and through it we are calling the function **message()**
- **main()** calls the function **message()**
- The activity of **main()** is temporarily suspended; it falls asleep while the **message()** function wakes up and goes to work.
- When the **message()** function runs out of statements to execute, the control returns to **main()**, which comes to life again and begins executing its code at the exact point where it left off.
- **main()** becomes the ‘calling’ function, whereas **message()** becomes the ‘called’ function.

Functions in C

- Any C program contains at least one function.
- If a program contains only one function, it must be **main()**.
- If a C program contains more than one function, then one (and only one) of these functions must be **main()**, because program execution always begins with **main()**.
- There is no limit on the number of functions that might be present in a C program.
- Each function in a program is called in the sequence specified by the function calls in **main()**.
- After each function has done its execution, control returns to **main()**. When **main()** runs out of function calls, the program ends.

Functions in C

- A function gets called when the function name is followed by a semicolon.

Example: `ABC();`

- A function is defined when function name is followed by a pair of braces in which one or more statements may be present.

Example: `ABC()`

`{`

`statement 1 ;`

`statement 2 ;`

`statement 3 ;`

`}`

Functions in C

- A function can be called any number of times.

```
main()  
{  
  message()  
  ; message()  
  ;  
}  
message()  
{  
  printf ( "\n Hello World!!" );  
}
```

Output:

```
Hello  
World!!  
Hello  
World!!
```

- The order in which the functions are defined in a program and the order in which they get called need not necessarily be same.

```
main()  
{  
  Message 1 ();  
  Message 2 ();  
}  
Message 2 ()  
{  
  printf ( "\n This is message 2" );  
}  
Message 1 ()  
{  
  printf ( "\n This is message 1" );  
}
```

Output:

```
This is message1  
This is message 2
```

Functions in C

- A function can be called from other function, but a function cannot be defined in another function.

```
main( )
{
printf ( "\n I am in main" ) ;
ABC( )
{
printf ( "\n I am in ABC" ) ;
}
}
```

- The above program code would be wrong, since **ABC()** is being defined inside another function, **main()**.

Definition and Types of Function

C functions can be classified into two categories:

- **Library functions:**

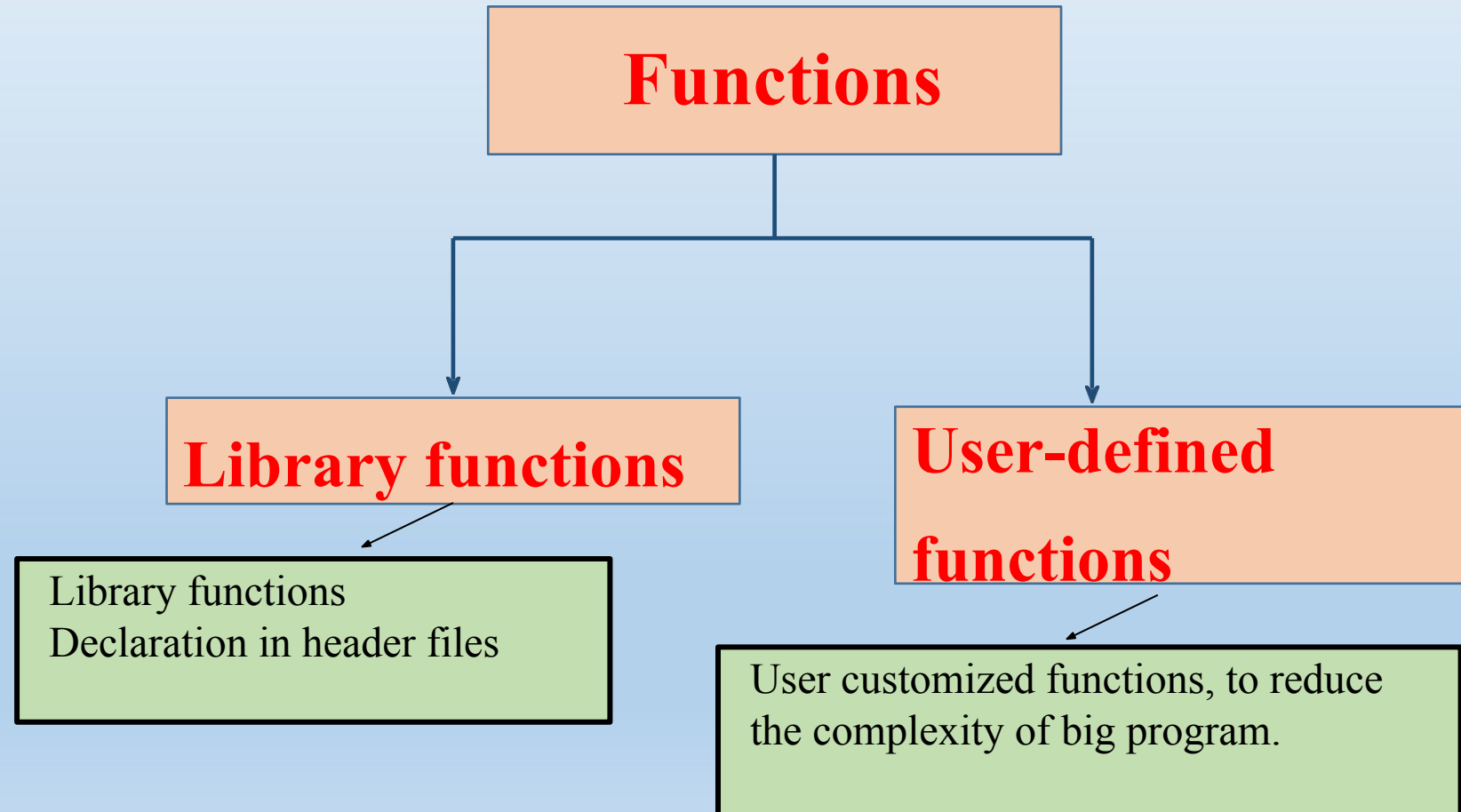
Library functions:

The standard library functions are built-in functions in C programming. These functions are defined in header files.

- **User-defined functions**

User-defined functions:

Functions created by the user are known as user-defined functions.



Types of Function: Example

Library functions:

- The printf() is a standard library function to send formatted output to the screen (display output on the screen).
- This function is defined in the stdio.h header file.
- The sqrt() function calculates the square root of a number.
- The function is defined in the math.h header file.

User-defined functions :

- ABC(), XYZ() etc.
- The procedure of calling both types of functions is exactly same.

Example 1: call more than one function

```
main( )
{
    printf ( "\nI am in main" );
    ABC();
    PQR();
    XYZ();
}
ABC(
)
{
    printf ( "\nI am in ABC" );
}
PQR(
)
{
    printf ( "\nI am in PQR" );
}
XYZ(
)
{
    printf ( "\nI am in XYZ" );
}
```

- Each function in a program is called in the sequence specified by the function calls in **main()**.
- There is no limit on the number of functions that might be present in a C program.

Output:

```
I am in main
I am in ABC
I am in PQR
I am in XYZ
```

Example 2: call more than one function

```
main( )
{
printf ( "\n I am in main" );
ABC();
printf ( "\n I am finally back in main" );
}
ABC ( )
{
printf ( "\n I am in ABC" );
PQR();
printf ( "\n I am back in ABC" );
}
PQR(
)
{
printf ( "\n I am in PQR" );
XYZ();
}
XYZ(
)
{
printf ( "\n I am in XYZ" );
```

- **main()** calls other functions, which in turn call still other functions.
- compiler always begins the program execution with **main()**, every function in a program must be called directly or indirectly by **main()**.

Output:

```
I am in main
I am in ABC
I am in PQR
I am in XYZ
I am back in ABC
I am finally back in main
```

Prototypes of Functions

A function prototype tells the compiler the type of data returned by the function, the number of parameters the function expects to receive, the types of the parameters, and the order in which these parameters are expected.

Syntax : `returntype functionName(type1 parameter1, type2 parameter2,...);`

Return type: Return type specifies the type of value(int, float, char, double) that function is expected to return to the program which called the function.

function name: it specifies the name of the function.

parameter list: The parameter list declares the type and number of arguments that the function expects when it is called.

terminating semicolon

Example: Prototypes of Functions

Example: 1

```
int maximum( int x, int y, int z ); /* function prototype */
```

This function prototype states that maximum takes three arguments of type int and returns a result of type int.

Example: 2

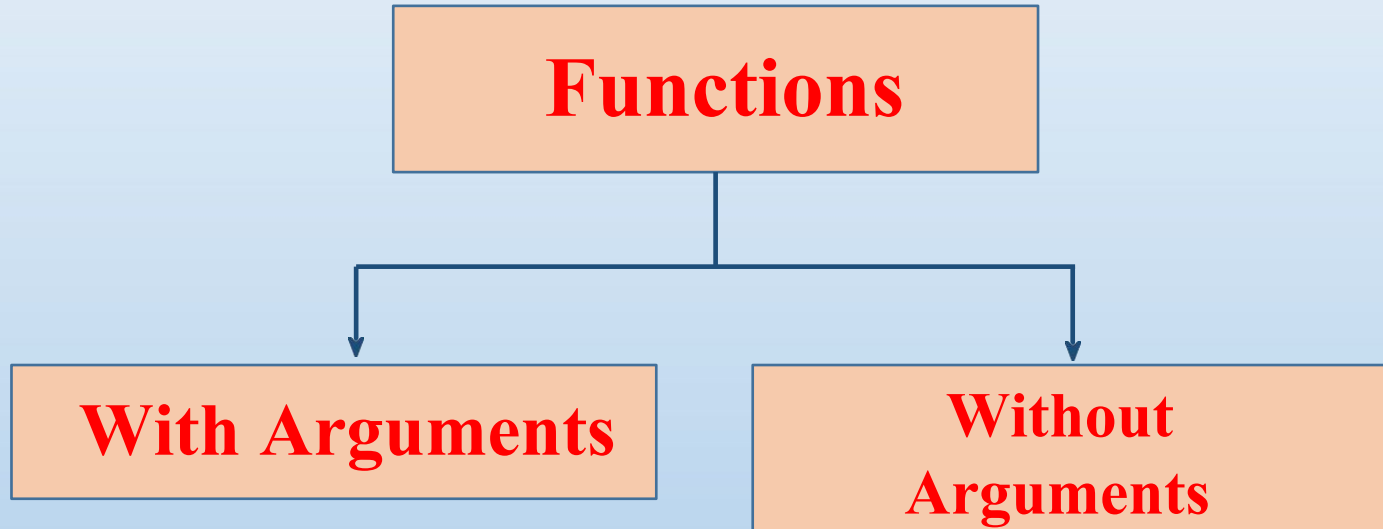
```
int area( int length, int breadth ) //function definition  
{  
    .... //function body  
}
```

Now, the corresponding prototype declaration of the above function is:

```
int area( int length, int breadth ); //function prototype
```

Passing Arguments to a function

Arguments are the values specified during the function call, for which the formal parameters are declared while defining the function.



- Declared and defined with parameter list
- Values for parameter passed during call.
- Example

Declaration: `int sum(int x, int y);`
Call: `Sum (10,20);`

- No parameter included
- No value passed during function call.
- Example.

Declaration: `int display( );`
Call: `display ( );`

Parameter Passing to functions

```
#include <stdio.h>
int main(void)
{
    int a = 10, b = 20;
    int m = max(a, b);
    printf("m is %d", m);
    return 0;
}

int max(int x, int y)
{
    if (x > y)
        return x;
    else
        return y;
}
```

Actual parameters:

- The parameters passed to function are called actual parameters.
- For example, in the given program 10 and 20 are actual parameters.

Formal parameters:

- The parameters received by function are called formal parameters.
- For example, in the above program x and y are formal parameters.

Advantages of user-defined function

- The program will be easier to understand, maintain and debug.
- Reusable codes that can be used in other programs.
- A large program can be divided into smaller modules. Hence, a large project can be divided among many programmers.
- Writing functions avoids rewriting the same code over and over.
- Using functions, it becomes easier to write programs and keep track of what they are doing.

Example-1 To find out square of a number

```
#include
```

```
<stdio.h> main( )
```

```
{
```

```
float a, b ;
```

```
printf ( "\nEnter any number "
```

```
) ; scanf ( "%f", &a ) ;
```

```
b = square ( a ) ;
```

```
printf ( "\nSquare of %f is %f", a, b
```

```
) ;
```

```
}
```

```
square ( float x )
```

```
{
```

```
float y ; y
```

```
= x * x ;
```

```
return ( y ) ;
```

```
}
```

Enter any number 3 Square of 3 is 9.000000 Enter any number 1.5 Square of 1.5 is 2.000000 Enter any number 2.5 Square of 2.5 is 6.000000

- The first of these answers is correct. But square of 1.5 is definitely not 2. Neither is 6 a square of 2.5
- This happened because any C function, by default, always returns an integer value
- How to make **square()** capable of returning a **float** value?

Example-1 To find out square of a number

```
#include <stdio.h>

main( )
{
float square ( float ) ;
float a, b ;
printf ( "\nEnter any number " ) ;
scanf ( "%f", &a ) ;
b = square ( a ) ;
printf ( "\nSquare of %f is %f", a, b )
;
}

float square ( float x )
{
float y ;
y = x * x
;
return ( y ) ;
}
```

Enter any number 1.5
Square of 1.5 is 2.250000
Enter any number 2.5
Square of 2.5 is 6.250000

- Now the expected answers i.e. 2.25 and 6.25 are obtained.
- The function **square()** must be declared in **main()** as **float square (float) ;**
- This statement is often called the prototype declaration of the **square()** function
- This means that **square()** is a function that receives a **float** and returns a **float**. We have done the prototype declaration in **main()** because we have called it from **main()**.

Example-2 Calculate the squares of the integers from 1 to 10

```
#include <stdio.h>

int square( int y ); /* function prototype */

int main( void )

{
    int x; /* counter */
    for ( x = 1; x <= 10; x++ ) {
        printf( "%d ", square( x )); /* function call */
    } /* end for */ printf(
        "\n" );

    return 0; /* indicates successful termination */
} /* end main */


int square( int y ) /* y is a copy of argument to function */
{
    return y * y; /* returns square of y as an int */
} /* end function square */
```

Output:

1 4 9 16 25 36 49 64 81 100

- Function square is **invoked** or **called** in main within the printf statement:
`printf( "%d ", square( x ) ); /* function call */`
- Function square receives a copy of the value of x in the parameter y.
- Then square calculates $y * y$.
- The result is passed back to function printf in main where square was invoked and printf displays the result.
- This process is repeated 10 times using the for repetition statement.
- The definition of function square shows that square expects an integer parameter y.
- `int square( int y );` is a **function prototype**

Example-3 Finding the maximum of three integers

```
#include <stdio.h>  
  
int maximum( int x, int y, int z ); /* function prototype */  
  
int main( void )  
{  
  int   number1;  
  int   number2;  
  int number3;  
  
  printf( "Enter three integers: " );  
  scanf( "%d%d%d", &number1, &number2, &number3 );  
  printf( "Maximum is: %d\n", maximum( number1, number2, number3 ));  
  return 0;  
}
```

Contd in next slide.....

Example-3 Finding the maximum of three integers

```
int maximum( int x, int y, int z )  
{  
    int max = x; if  
    ( y > max ) {  
        max = y;  
    }  
    if ( z > max ) {  
        max = z;  
    }  
    return max;  
}
```

Enter three integers: **11 83 12**

Maximum is: 83

Enter three integers: **83 11 12**

Maximum is: 85

Enter three integers: **12 11 83**

Maximum is: 85

Example-4 Program to Check Prime Number

```
#include<stdio.h>
int check_prime(int);
main()
{
    int n, result;
    printf("Enter an integer to check whether it is prime or not.\n");
    scanf("%d",&n);
    result = check_prime(n); if
    ( result == 1 )
        printf("%d is prime\n", n);
    else
        printf("%d is not prime\n", n);
    return 0;
}
```

Contd in next
slide.....

Example-4 Program to Check Prime Number

```
int check_prime(int a)
{
    int c;
    for ( c = 2 ; c <= a - 1 ; c++ )
    {
        if ( a%c == 0 ) return 0;
    }
    return 1;
}
```

Enter a positive integer: 29

29 is a prime number.